"Pixie and Dixie vs. Mr. AI": One-Pixel Attack

Copyright © 2019 Liran Ma, Texas Christian University; Zhipeng Cai, Georgia State University; Wei Cheng, University of Washington Tacoma

The development of this document is partially funded by the National Science Foundation's CyberTraining program under award No. OAC-1829553. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at `http://www.gnu.org/licenses/fdl.html`.

Introduction

The purpose of this lab is to understand and exploit vulnerabilities of deep neural networks used in Internet of Things (IoT). Image data and a trained deep neural network model are typically used to classify an image (e.g., facial recognition for authentication) on mobile and/or IoT devices. In this lab, you will prompted to pinpoint a vulnerable pixel such that changing this pixel will result in wrong classification prediction, which may confuse relevant security modules.

Contents

1 Backgrounds **2**

2 Lab Preparation **3**

3 Hand craft one-pixel attack **4**

4 Automatic One-pixel Attack **7**

5 Discussion Questions **9**

6 General Questions **10**

1 Backgrounds

1.1 Neural Networks

Image classification is the process of taking an input (like an image) and giving a class (like "cat") or a probability that the input is a particular class ("the confidence of cat is 90%"). Among various models that designed for image classification, neural network outperform other techniques significantly. Neural Network is a computational model that works in a similar way to the neurons in the human brain. Each neuron takes an input, performs some operations then passes the output to the following neuron.

1.1.1 Deep Residual Neural Network

Normally, deeper network means higher accuracy achievement. However, deeper neural networks are more difficult to train. Kaiming He *et al.*[**?**] proposed Deep Residual Neural Network (ResNet), which is easy to optimize, and can easily enjoy accuracy gains from greatly increased depth. In this lab, we will use ResNet as the target neural network.

1.2 Cifar-10 Dataset

In our lab, we use cifar10 as our dataset (`https://www.cs.toronto.edu/~kriz/cifar.html`). The CIFAR-10 is a labeled colour image dataset. It has total 60000 32×32 images. The images in CIFAR-10 are low-resolution (32x32), which means the dataset can allow researchers to quickly try different algorithms to see what works.

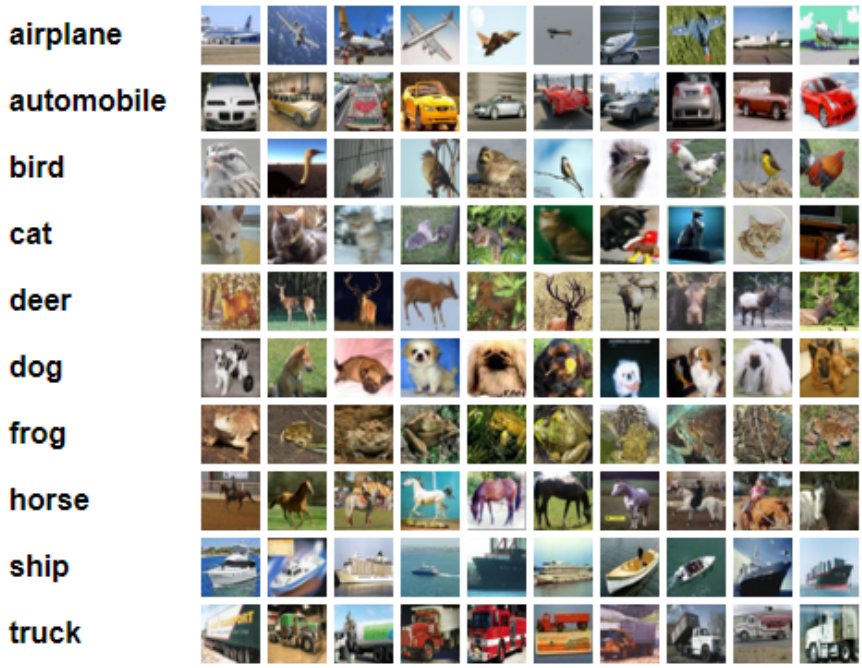

Figure 1: Cifar 10 Dataset

2 Lab Preparation

We will do the lab project on Google's Colaboratory `https://colab.research.google.com`. Everyone can log into the Colab with your own google account. In this section, we will simply introduce the basic functions of Google Golab platform, which will be used in our experiments. We only show the necessary steps and setup commands related to open and create a notebook on Colab, more details about concrete implementation of codes will be displayed in following sections. If you are very interested in deep learning applications, you can learn more from Youtube link `https://www.youtube.com/watch?v=inN8seMm7UI` about how to use Colab to train more machine learning models.

2.1 New a Notebook to Start

First of all, we need to new a notebook for code programming in Colab, shown in Figure 2. You can name it as "One-Pixel Attack" (OPA for short) or any other name you want to specify this notebook file.

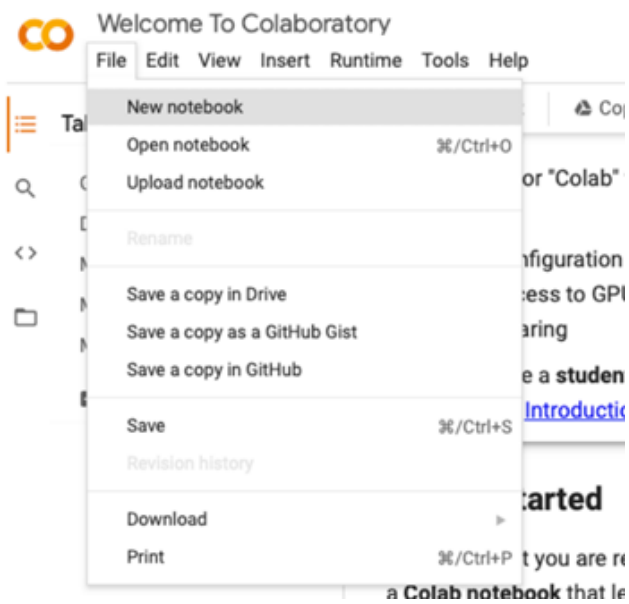

Figure 2: New a Notebook

2.2 Set Reference Folder

In this lab we will import code from external file. Thus, after we enter the new notebook, we need to set reference folder. We can go "File->Locate in Drive" to open the path of the where notebook locates. Then create a folder there then put all the attached file in it. Then we can go back to notebook input the following code. Remember that you should replace /OPALab/ with your own folder's name.

```
import sys
from google.colab import drive
drive.mount('/content/drive')
sys.path.append('/content/drive/My Drive/Colab Notebooks/OPALab')
```

```
Mounted at /content/drive
```

Figure 3: Set Reference Path

3 Hand craft one-pixel attack

3.1 Data & Model Preparation

We use deep learning framework TensorFlow for the implementation of one-pixel attack. The data we use to illustrate this experiment can be downloaded from TensorFlow libraty by keras.datasets. Necessary code is below:

```
import pickle
import numpy as np
import pandas as pd
import matplotlib
from keras.datasets import cifar10
from keras import backend as K
from networks.resnet import ResNet
from differential_evolution import differential_evolution
import helper
matplotlib.style.use('ggplot')
np.random.seed(100)
```

Then we need to split the data into train dataset and test dataset, set class_names and new a model object using given function:

```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog',
    'horse', 'ship', 'truck']
resnet = ResNet()
```

Now we can try to use function plot_image to plot images from the dataset:

```
image_id = 99 # Image index in the test set
helper.plot_image(x_test[image_id])
```

3.2 Predict an Image's Class

In the imported model, we can use method predict_one to predict an image's classification on given model. The predict_one function will give the image's confidence on all the classes. Following code will give the confidence in true class of the given image.

```
image_id = 11
true_class = y_test[image_id, 0]
cof = resnet.predict_one(x_test[image_id])[true_class]
helper.plot_image(x_test[image_id])
print("the confidence in true class " + class_names[true_class] + " is " + str(cof))
```

```
the confidence in true class truck is 1.0
```

Figure 4: Predicting an Image

3.3 Perturb the Pixels in an Image

A pixel in an image can be described as $\mathbf{x} = (x, y, r, g, b)$, where x, y are the coordinates of the pixel from 0 to 31 (for cifar10), and r, g, b are the red, green, and blue values from 0 to 255. You need a strategy to locate the coordinate x and y, and determine the value change of r, g and b.
The image is represented by a 2-dimensional array. To modify the color the pixel locates at `(x_pos, y_pos)` of image `img` to color `rgb`, you can use: `img[x_pos, y_pos] = rgb`
Here we give the function to perturb images:

```
def perturb_image(xs, img):
    # If this function is passed just one perturbation vector,
    # pack it in a list to keep the computation the same
    if xs.ndim < 2:
        xs = np.array([xs])

    # Copy the image n == len(xs) times so that we can
    # create n new perturbed images
    tile = [len(xs)] + [1]*(xs.ndim+1)
    imgs = np.tile(img, tile)

    # Make sure to floor the members of xs as int types
```

```
    xs = xs.astype(int)

    for x,img in zip(xs, imgs):
        # Split x into an array of 5-tuples (perturbation pixels)
        # i.e., [[x,y,r,g,b], ...]
        pixels = np.split(x, len(x) // 5)
        for pixel in pixels:
            # At each pixel's x,y position, assign its rgb value
            x_pos, y_pos, *rgb = pixel
            img[x_pos, y_pos] = rgb

    return imgs
```

After we have the function of perturbing the images, we can try to modify an image:

```
image_id = 91
pixel = np.array([16, 16, 255, 255, 0]) # pixel = x,y,r,g,b
image_perturbed = perturb_image(pixel, x_test[image_id])[0]
helper.plot_image(image_perturbed)
```

Figure 5: Perturb a Pixel on Cat

Then we get a perturbed image.

3.4 Execute One-pixel Attack

In most case, changing one pixel will not cause the classification confidence of an image change a lot. However, in some case, it may change the image's confidence greatly. Here we give an example:

```
image_id = 384
pixel = np.array([16, 13, 25, 48, 156])
true_class = y_test[image_id, 0]
prior_confidence = resnet.predict_one(x_test[image_id])[true_class]
perturbed_image = perturb_image(pixel, x_test[image_id])
confidence = resnet.predict_one(perturbed_image)[true_class]
```

```
    print('Confidence in', class_names[true_class], ' was', prior_confidence, 'before
        perturbation')
    print('After perturbation, confidence changes to ', confidence)

    helper.plot_image(x_test[image_id])
    helper.plot_image(perturbed_image)
```

```
Confidence in bird  was 0.7066177 before perturbation
After perturbation, confidence changes to  0.00018887446
```

Figure 6: One-pixel Attack on a Bird

Now, we successfully made a one-pixel attack on this bird image.

4 Automatic One-pixel Attack

The attack problem can be formulated as an optimization problem, aiming at minimizing the confidence of the correct class. In this lab, we use an algorithm called differential evolution.
To determine whether the modification on a pixel is an effective perturbation, we need a function to know the image's confidence on the true label.

```
def predict_classes(xs, img, target_class, model, minimize=True):
    # Perturb the image with the given pixel(s) x and get the prediction of the model
    imgs_perturbed = perturb_image(xs, img)
    predictions = model.predict(imgs_perturbed)[:,target_class]
    # This function should always be minimized, so return its complement if needed
    return predictions if minimize else 1 - predictions
```

Then we need a function to judge whether the classification by model on the image has been successfully changed.

```
def attack_success(x, img, target_class, model, targeted_attack=False, verbose=False):
    # Perturb the image with the given pixel(s) and get the prediction of the model
    attack_image = perturb_image(x, img)

    confidence = model.predict(attack_image)[0]
    predicted_class = np.argmax(confidence)
```

```
    # If the prediction is what we want (misclassification or
    # targeted classification), return True
    if verbose:
        print('Confidence:', confidence[target_class])
    if ((targeted_attack and predicted_class == target_class) or
        (not targeted_attack and predicted_class != target_class)):
        return True
```

Now we can use differential evolution to implement our attack:

```
def attack(img_id, model, target=None, pixel_count=1,
           maxiter=75, popsize=400, verbose=False):
    # Change the target class based on whether this is a targeted attack or not
    targeted_attack = target is not None
    target_class = target if targeted_attack else y_test[img_id, 0]

    # Define bounds for a flat vector of x,y,r,g,b values
    # For more pixels, repeat this layout
    bounds = [(0,32), (0,32), (0,256), (0,256), (0,256)] * pixel_count

    # Population multiplier, in terms of the size of the perturbation vector x
    popmul = max(1, popsize // len(bounds))

    # Format the predict/callback functions for the differential evolution algorithm
    def predict_fn(xs):
        return predict_classes(xs, x_test[img_id], target_class,
                               model, target is None)

    def callback_fn(x, convergence):
        return attack_success(x, x_test[img_id], target_class,
                              model, targeted_attack, verbose)

    # Call Scipy's Implementation of Differential Evolution
    attack_result = differential_evolution(
        predict_fn, bounds, maxiter=maxiter, popsize=popmul,
        recombination=1, atol=-1, callback=callback_fn, polish=False)

    # Calculate some useful statistics to return from this function
    attack_image = perturb_image(attack_result.x, x_test[img_id])[0]
    prior_probs = model.predict_one(x_test[img_id])
    predicted_probs = model.predict_one(attack_image)
    predicted_class = np.argmax(predicted_probs)
    actual_class = y_test[img_id, 0]
    success = predicted_class != actual_class
```

```
    cdiff = prior_probs[actual_class] - predicted_probs[actual_class]

    # Show the best attempt at a solution (successful or not)
    helper.plot_image(attack_image, actual_class, class_names, predicted_class)

    return [model.name, pixel_count, img_id, actual_class, predicted_class, success,
        cdiff, prior_probs, predicted_probs, attack_result.x]
```

Then we can try to apply the attack on images:

```
image_id = 102
pixels = 1 # Number of pixels to attack
model = resnet

_ = attack(image_id, model, pixel_count=pixels, verbose=True)
```

```
Confidence: 0.96962196
Confidence: 0.87273866
Confidence: 0.6955495
Confidence: 0.42089856
```

True: frog
Predicted: dog

Figure 7: One-pixel Attack on a Frog Image

5 Discussion Questions

5.0.1 What is the largest problem you find with this attack?

5.0.2 As a shared model provider, how could you prevent this attack?

5.0.3 Once a dataset has been poisoned, what can you do to avoiding give a wrong result on a benign model

6 General Questions

6.0.1 How long did it take you to complete this lab?

6.0.2 Are there any corrections that need to be made to this lab?